Đại học Quốc gia Thành phố Hồ Chí Minh
Trường đại học Khoa học Tự nhiên
Khoa Công nghệ Thông tin

—

Môn: An ninh Mạng

Project 1:
Security Smart Contract

Sinh viên 21127322 - Hoàng Xuân Khôi
21127643 - Nguyễn Chí Lương

Giảng viên Lê Giang Thanh
Nguyễn Thanh Quân
Lê Hà Minh

TPHCM, 21.6.2024

Mục lục

1. Các mục đã hoàn thành.....2
2. Kết quả thực hiện.....3
a. Câu 1: Unstoppable.....3
b. Câu 2: Naive Receiver.....4
c. Câu 3: Truster.....5
d. Câu 4: Side Entrance.....6
e. Câu 5: The Rewarder.....7
f. Câu 6: Selfie.....8
g. Câu 7: Compromised.....10
h. Câu 8: Puppet.....11
i. Câu 9: Puppet V2.....12
j. Câu 10: Free Rider.....13
k. Câu 11: Backdoor.....15
l. Câu 12: Climber.....16
n. Câu 14: Puppet V3.....18
o. Câu 15: ABI Smuggling.....19
p. Câu 16: Shards.....20
q. Câu 17: Curvy Puppet.....22
r. Câu 18: Withdrawal.....23

1. Các mục đã hoàn thành

Câu	Thực hiện	Hoàn thành
Câu 1	Nguyễn Chí Lương	Hoàn thành
Câu 2	Hoàng Xuân Khôi	Hoàn thành
Câu 3	Nguyễn Chí Lương	Hoàn thành
Câu 4	Hoàng Xuân Khôi	Hoàn thành
Câu 5	Nguyễn Chí Lương	Hoàn thành
Câu 6	Hoàng Xuân Khôi	Hoàn thành
Câu 7	Nguyễn Chí Lương	Hoàn thành
Câu 8	Hoàng Xuân Khôi	Hoàn thành
Câu 9	Nguyễn Chí Lương	Hoàn thành
Câu 10	Hoàng Xuân Khôi	Hoàn thành
Câu 11	Nguyễn Chí Lương	Hoàn thành
Câu 12	Hoàng Xuân Khôi	Hoàn thành
Câu 13	Nguyễn Chí Lương	Hoàn thành
Câu 14	Hoàng Xuân Khôi	Hoàn thành
Câu 15	Nguyễn Chí Lương	Hoàn thành
Câu 16	Hoàng Xuân Khôi	Hoàn thành
Câu 17	Nguyễn Chí Lương	Hoàn thành
Câu 18	Hoàng Xuân Khôi	Hoàn thành

2. Kết quả thực hiện

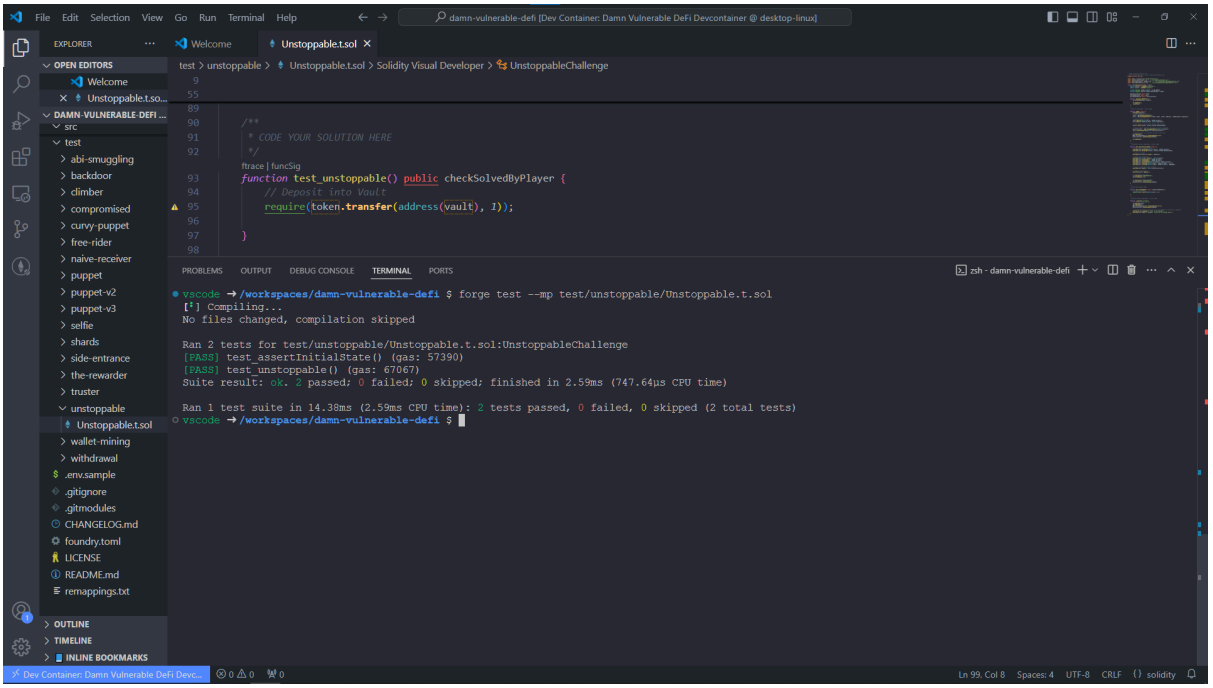
a. Câu 1: Unstoppable

i. Mô tả lỗ hổng

- Có một kho token DVT chứa 1 triệu token. Nó cung cấp dịch vụ Flash Loan mà không tính phí trong một khoảng thời gian nhất định(hết thời gian thì không cho Flash Loan nữa).
- Có một contract để giám sát tính khả dụng của tính năng Flash Loan
- Với 10 DVT trong ví. Hãy tìm cách để khiến cho dịch vụ Flash Loan của kho ngưng hoạt động

ii. Khai thác lỗ hổng

- Điều kiện chiến thắng challenge này là:
  + Khiến cho nó không cung cấp dịch vụ Flash Loan nữa
- Cách làm:
  + Chuyển trực tiếp token vào trong kho
  + Điều này sẽ khiến cho shares != token
  + unfinished



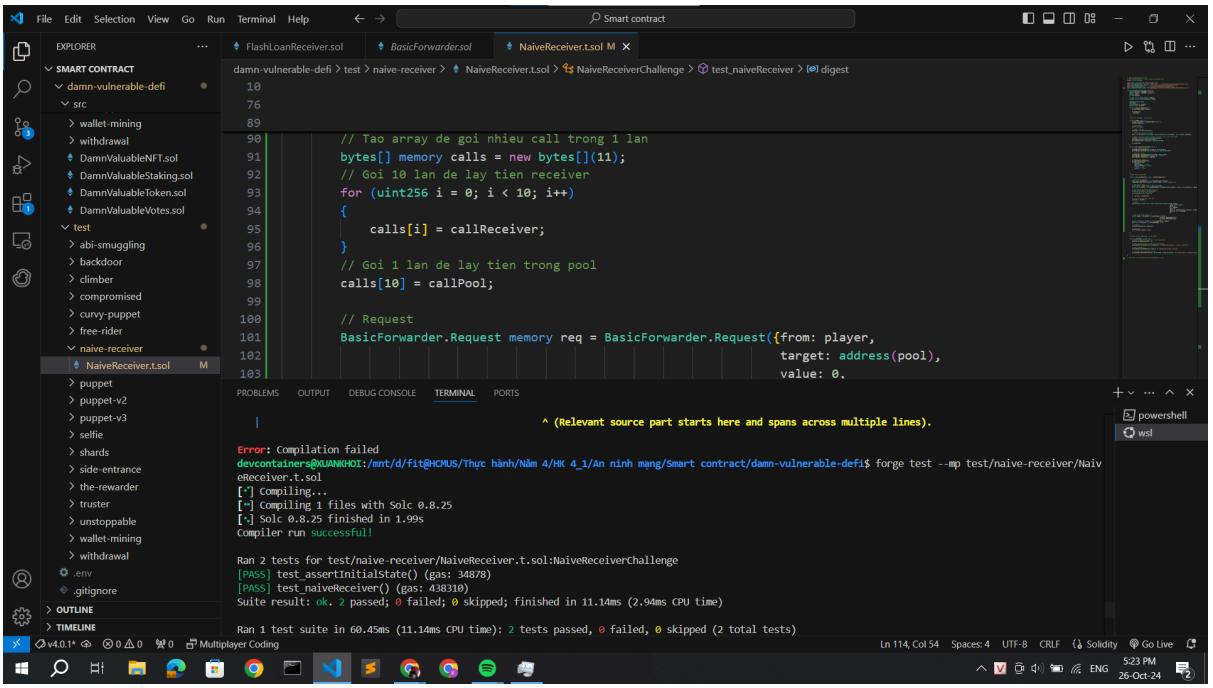
b. Câu 2: Naive Receiver

i. Mô tả lỗ hổng

- Đề bài cho ta biết:
  + Người cho vay có 100 WETH
  + Người vay có 10 WETH
- Chúng ta cần phải tìm cách lấy tất cả số WETH từ cả người vay và người cho vay
- Trong hàm **onFlashLoan,** ta thấy nó không kiểm tra người vay tiền (msg.sender)

ii. Khai thác lỗ hổng

- Điều kiện chiến thắng challenge này là:
  + Chỉ có thể thực hiện tối đa 2 giao dịch
  + Rút hết WETH
  + Chuyển toàn bộ về tài khoản Recovery
- Cách làm:
  + Sử dụng multicall để thực hiện nhiều giao dịch trong một lần call.
  + Thực hiện 10 lần flash loan, với địa chỉ người nhận là receiver
  + Tạo chữ ký cho yêu cầu để xác thực giao dịch.
- Hoàn thành challenge



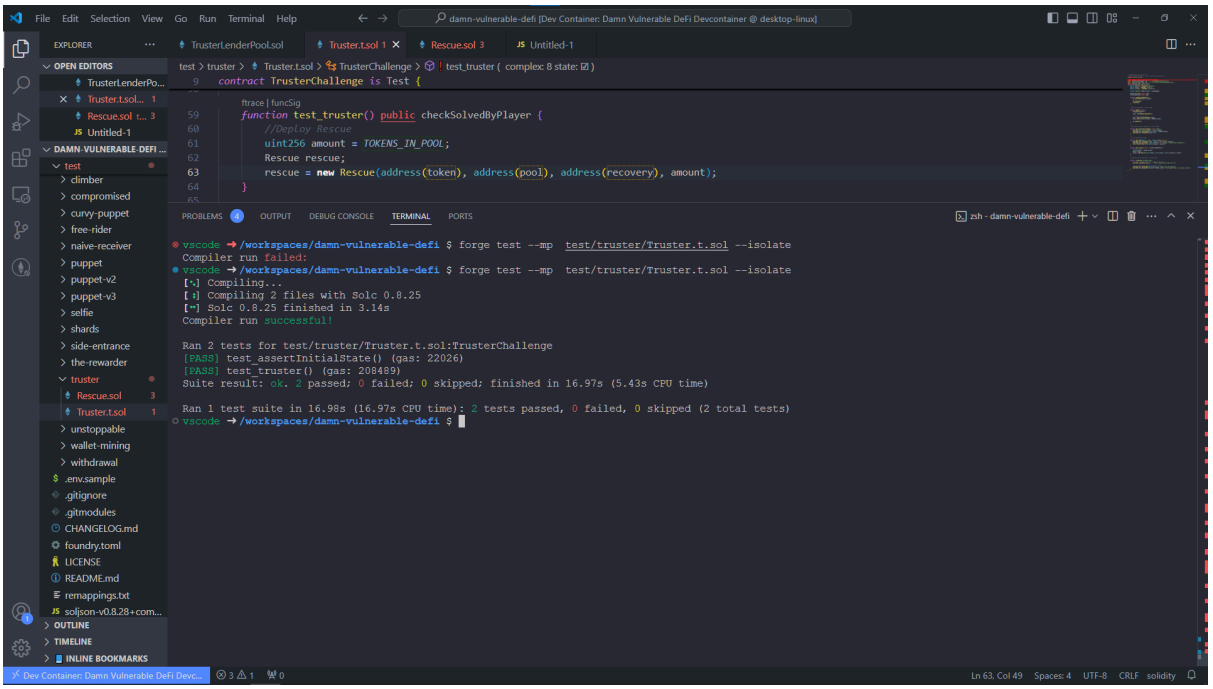
c. Câu 3: Truster

i. Mô tả lỗ hổng

- Có một dịch vụ Flash Loan đang hoạt động, cung cấp dịch vụ không mất phí
- Nó đang chứa 1 triệu DVT token. Người chơi không có gì cả
- Hãy giải cứu tất cả token trong một transaction. Chuyển vào recovery account

ii. Khai thác lỗ hổng

- Vấn đề nằm ở việc khi gọi hàm **flashloan** thì người gọi được cho phép gọi một hàm bất kỳ
- Bằng cách tạo một contract, với constructor sẽ thực hiện tuần tự các thao tác sau:
  + Gọi hàm flashloan để mượn 0 token với tham số là hàm Cho phép chính nó quyền được sử dụng 1 Triệu token.
  + Chuyển token vào recovery account



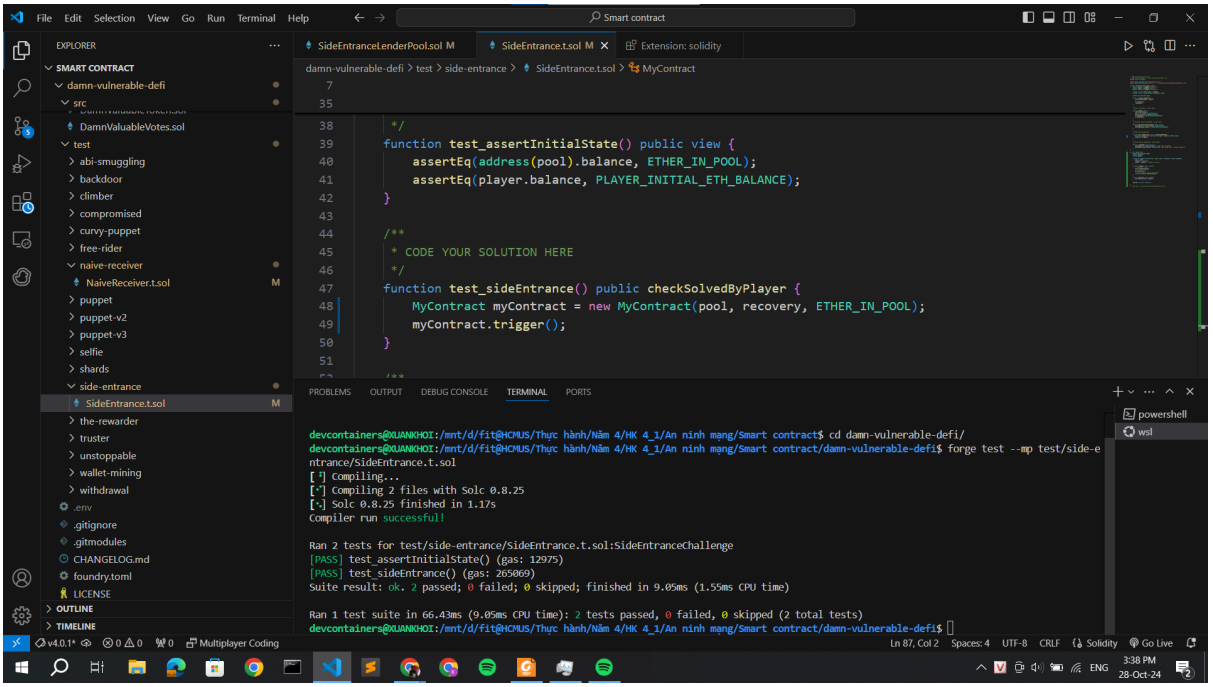
d. Câu 4: Side Entrance

i. Mô tả lỗ hổng

- Hiện tại pool có 1000 ETH và chúng ta có thể rút
- Pool cung cấp các khoản vay nhanh miễn phí bằng cách sử dụng số ETH đã gửi để quảng bá hệ thống của họ.
- Vượt qua challenge bằng cách rút toàn bộ ETH và gửi vào tài khoản recovery
- Ở hàm **flashLoan** ta có thể thấy **FlashLoanEtherReceiver** sẽ triển khai hàm **execute** trong contract
- Điều này có nghĩa là chúng ta có thể tạo ra một contract độc hại và định nghĩa hàm **execute** trong đó.

ii. Khai thác lỗ hổng

- Điều kiện chiến thắng:
  + Không còn tiền trong pool
  + Toàn bộ tiền được rút vào recocery account
- Cách làm:
  + Vì chúng ta cần phải để cho hàm **flashLoan** gọi hàm **execute** trong **FlashLoanEtherReceiver** nên ta có lẽ phải viết 1 contract mới
  + Đầu tiên, ta sẽ **flashLoan** để vay 1 lượng **amount** (= 1000 ETH)
  + Khi gọi **flashLoan**, pool sẽ gửi lượng ETH đó đến MyContract và gọi hàm execute của MyContract
  + Hàm **execute** sẽ được kích hoạt, trong đó lượng ETH đã vay được nạp lại vào pool thông qua hàm **deposit**. Việc nạp lại này giúp cập nhật **balance[msg.sender]** trong pool, ghi nhận rằng user đã có số dư 1000 ETH
  + Sau khi **execute** hoàn tất, hàm **withdraw** sẽ được gọi để rút toàn bộ số ETH đã tích lũy
  + Cuối cùng, chuyển toàn bộ ETH đến tài khoản **recovery**
- Hoàn thành challenge



e. Câu 5: The Rewarder

i. Mô tả lỗ hổng

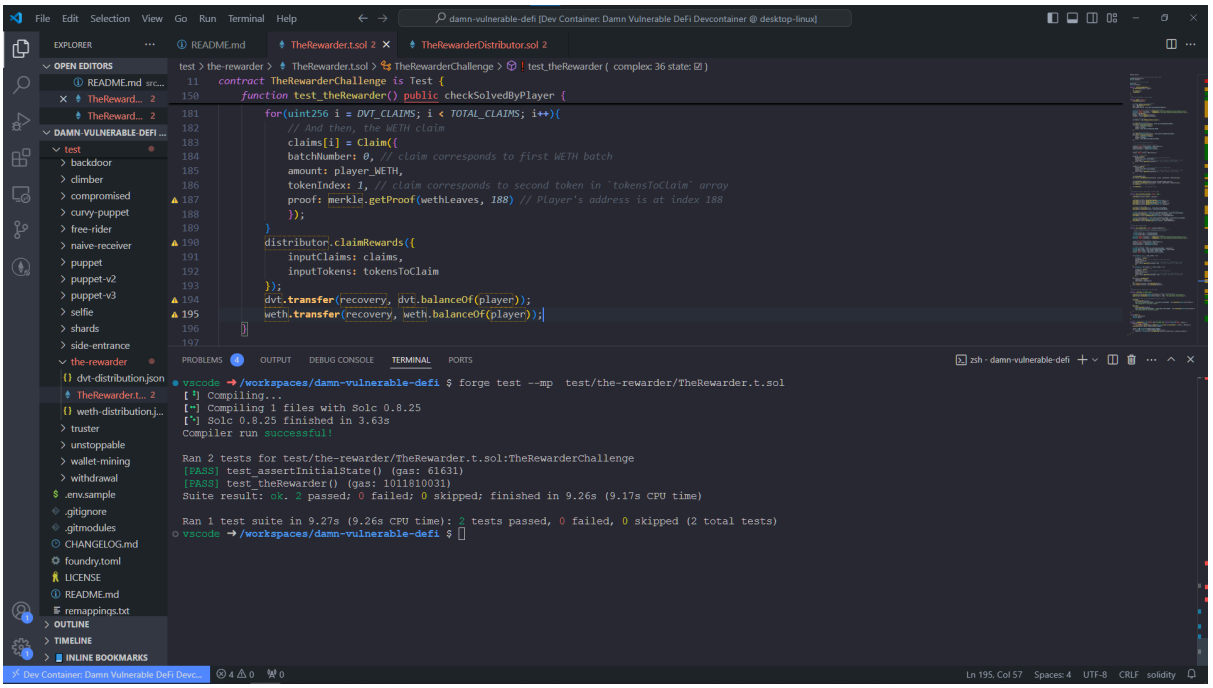
- Một contract đang phát phần thưởng dưới dạng Damn Valuable Tokens và WETH
- Để nhận thưởng, người dùng phải chứng minh họ nằm trong nhóm được nhận thưởng
- Contract được tối ưu để nhận nhiều token trong một transaction
- Giải cứu càng nhiều tiền càng tốt từ người phát thưởng, chuyển tất cả số assets được phục hồi đến tài khoản recovery.

ii. Khai thác lỗ hổng

- Điều kiện chiến thắng:
  + Lấy nhiều tiền nhất có thể
  + Chuyển tất cả tiền vào recovery account
- Cách làm:
  + Tìm address của player (0x44E97aF4418b7a17AABD8090bEA0A471a366305C)
  + Tính số lần cần nhận của mỗi loại Token để cố gắng lấy tất cả tiền
  + Ta thấy rằng hàm **claimRewards** có lỗi ở chỗ nó chỉ đánh dấu ở lần cuối cùng nếu chúng ta gửi các yêu cầu lấy cùng một loại token.

+ Thế nên trong mảng **claims**, chúng ta tạo liên tục các yêu cầu lấy DVT trước rồi tới yêu cầu lấy WETH với số phần tử bằng với số lần nhận của token đó

- Hoàn thành Challenge:



f. Câu 6: Selfie

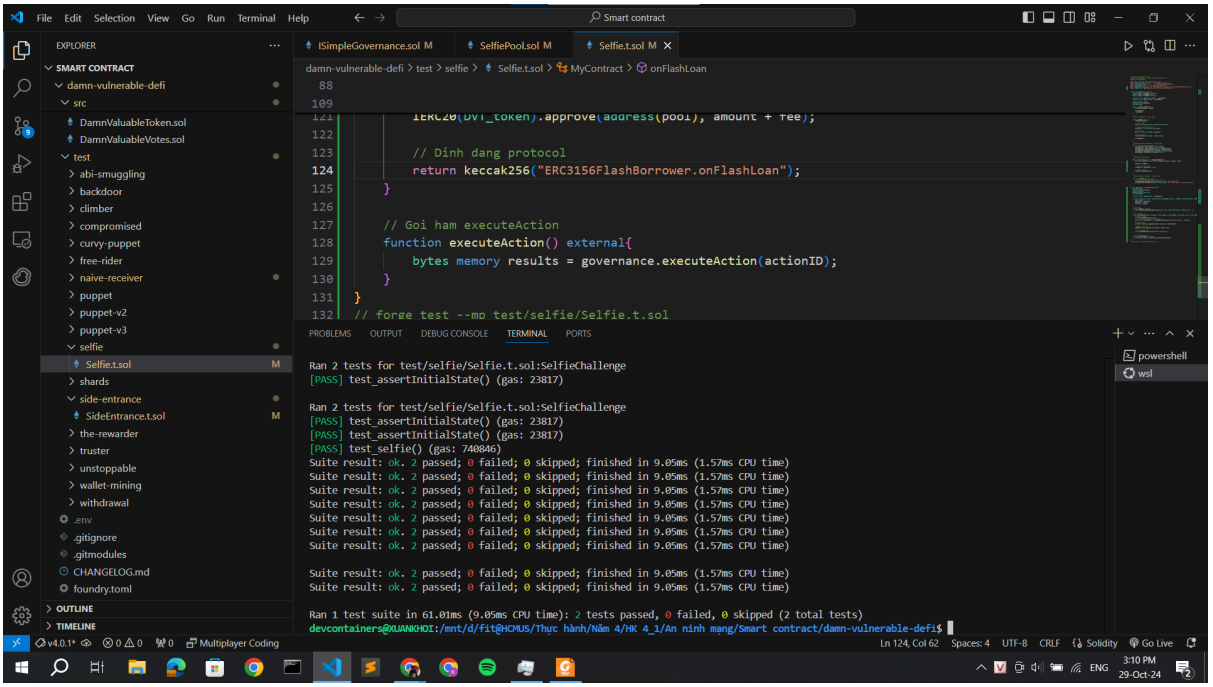
i. Mô tả lỗ hổng

- Hiện tại, người dùng không có DVT trong tài khoản
- Pool có 1,5 triệu token DVT có nguy cơ bị tấn công
- Cần chiếm toàn bộ số token trong pool và gửi vào tài khoản recovery
- Đọc các contract ta có thể thấy hàm **_hasEnoughVotes** có một vấn đề logic khi chỉ kiểm tra vote tại thời điểm gọi **queueAction**, mà không kiểm tra lại trong **executeAction**.
- Do lỗi này, ta có thể lợi dụng theo các bước sau:
  + Mượn quyền biểu quyết tạm thời thông qua flash loan từ **SelfiePool**, giúp tăng số dư token để tạm thời có đủ quyền biểu quyết.
  + **queueAction()**: Đề xuất một hành động độc hại với mục đích sử dụng **emergencyExit** của **SelfiePool**, nhắm đến việc chuyển tất cả tài sản trong SelfiePool vào **recovery**.

+ **executeAction()**: Do không có yêu cầu phải duy trì quyền biểu quyết tại thời điểm thực thi, chúng ta có thể thực thi hành động độc hại này và kích hoạt **emergencyExit** mà không cần giữ lại số token biểu quyết.

ii. Khai thác lỗ hổng

- Điều kiện chiến thắng:
  + Không còn tiền trong pool
  + Toàn bộ tiền được rút vào recocery account
- Cách làm:
  + Để bắt đầu flash loan, lấy số tiền từ SelfiePool vào contract của mình.
  + Sau khi flash loan được thực hiện, phương thức **onFlashLoan** sẽ được gọi. Tại đây, ta sẽ thực hiện các bước để bỏ phiếu độc hại.
  + Sử dụng vm.warp để giả lập việc thời gian đã trôi qua 2 ngày
  + Gọi **executeAction** để thực thi hành động mà ta đã bỏ phiếu trước đó, cụ thể là thực hiện **emergencyExit**.
- Hoàn thành challenge:



g. Câu 7: Compromised

i. Mô tả lỗ hông

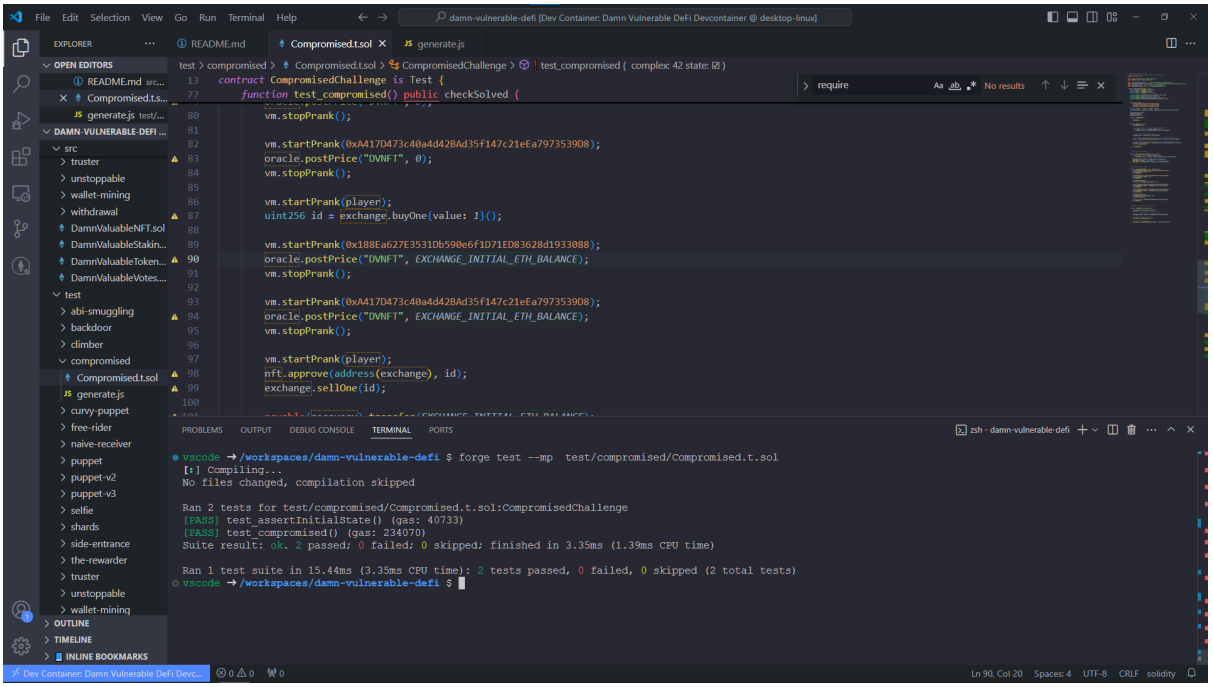
- Khi đang khám phá một dịch vụ web, bạn bỗng nhiên nhận được một phản hồi lạ từ server

```
HTTP/2 200 OK
content-type: text/html
content-language: en
vary: Accept-Encoding
server: cloudflare

4d 48 67 33 5a 44 45 31 59 6d 4a 68 4d 6a 5a 6a 4e 54 49 7a 4e 6a 67 7a 59 6d 5a 6a 4d 32 52
6a 4e 32 4e 6b 59 7a 56 6b 4d 57 49 34 59 54 49 33 4e 44 51 30 4e 44 63 31 4f 54 64 6a 5a 6a
52 6b 59 54 45 33 4d 44 56 6a 5a 6a 5a 6a 4f 54 6b 7a 4d 44 59 7a 4e 7a 51 30

4d 48 67 32 4f 47 4a 6b 4d 44 49 77 59 57 51 78 4f 44 5a 69 4e 6a 51 33 59 54 59 35 4d 57 4d
32 59 54 56 6a 4d 47 4d 78 4e 54 49 35 5a 6a 49 78 5a 57 4e 6b 4d 44 6c 6b 59 32 4d 30 4e 54
49 30 4d 54 51 77 4d 6d 46 6a 4e 6a 42 69 59 54 4d 33 4e 32 4d 30 4d 54 55 35
```

- Một dịch vụ giao dịch đang bán một thứ gọi là “DVNFT” với giá 999 ETH(rất đắt)
- Giá này được thu thập được từ một on-chain oracle, dựa trên ba nguồn được tin tưởng: 0x188...088, 0xA41...9D8 and 0xab3...a40.
- Với 0.1 ETH, hãy giải cứu tất cả ETH có trên dịch vụ giao dịch đó. Chuyển vào recovery account

ii. Khai thác lỗ hổng

- Thông tin nhận được có vẻ ở dạng Hex. Thử convert sang ASCII, ta được một chuỗi có vẻ ở dạng Base64. Decode nó, ta được:
  + 0x7d15bba26c523683bfc3dc7cdc5d1b8a2744447597cf4da1705cf6c993063744
  + 0x68bd020ad186b647a691c6a5c0c1529f21ecd09dcc45241402ac60ba377c4159
- Có vẻ nó là Private Key, từ đó ta tạo được hai Wallet(bằng file generate.js) có Address là:
  + 0x188Ea627E3531Db590e6f1D71ED83628d1933088
  + 0xA417D473c40a4d42BAd35f147c21eEa7973539D8
- Ta thấy nó trùng với hai trong ba địa chỉ của các nguồn được tin tưởng
- Bây giờ, ta sẽ đóng giả làm hai nguồn được tin tưởng trên, đổi giá tiền của **DVNFT** xuống 0. Sau đó Player mua nó. Rồi ta đổi lại thành 999 ETH để player bán và nhận về 999 ETH



h. Câu 8: Puppet

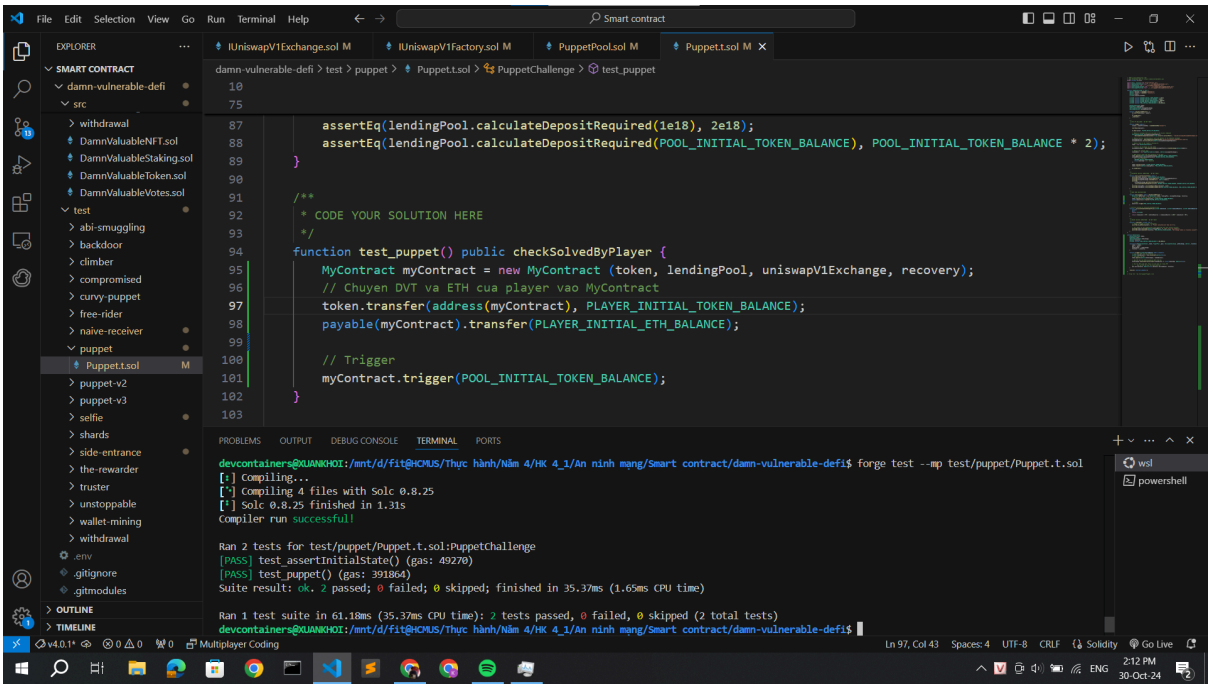
i. Mô tả lỗ hổng

- Bài toán cung cấp một lending pool nơi người dùng có thể vay Damn Valuable Tokens (DVT). Để có thể vay, người dùng cần đặt cọc lượng ETH có giá trị gấp đôi lượng vay.
- Hiện tại, pool có 100.000 DVT trong thanh khoản.
- Ngoài ra, có một thị trường DVT đang hoạt động trên sàn Uniswap v1 cũ, hiện có 10 ETH và 10 DVT trong thanh khoản.
- Contract **PuppetPool** cho phép người dùng vay token DVT từ pool với điều kiện thế chấp gấp đôi giá trị số token muốn vay bằng ETH.
- Trong hợp đồng này, một điểm đáng chú ý là giá token DVT tính dựa trên giá của cặp thanh khoản **Uniswap V1 (uniswapPair)**, điều này mở ra khả năng thao túng giá của DVT để giảm lượng ETH cần thế chấp cho khoản vay.
- Mục tiêu là làm cho giá của DVT trở nên rẻ hơn thông qua việc thao túng cặp thanh khoản Uniswap

ii. Khai thác lỗ hổng

- Đầu tiên ta sẽ gọi **tokenToEthTransferInput** để bán DVT và nhận ETH

- ETH trong pool giảm đi, DVT tăng lên, dẫn đến giá của DVT (tính theo ETH) giảm mạnh.
- Sau khi giá DVT đã bị giảm mạnh, **PuppetPool** sẽ tính toán lượng ETH yêu cầu cho tài sản thế chấp thấp hơn nhiều để có thể vay toàn bộ số DVT trong pool thông qua hàm **borrow**
- Hoàn thành challenge:



i. Câu 9: Puppet V2

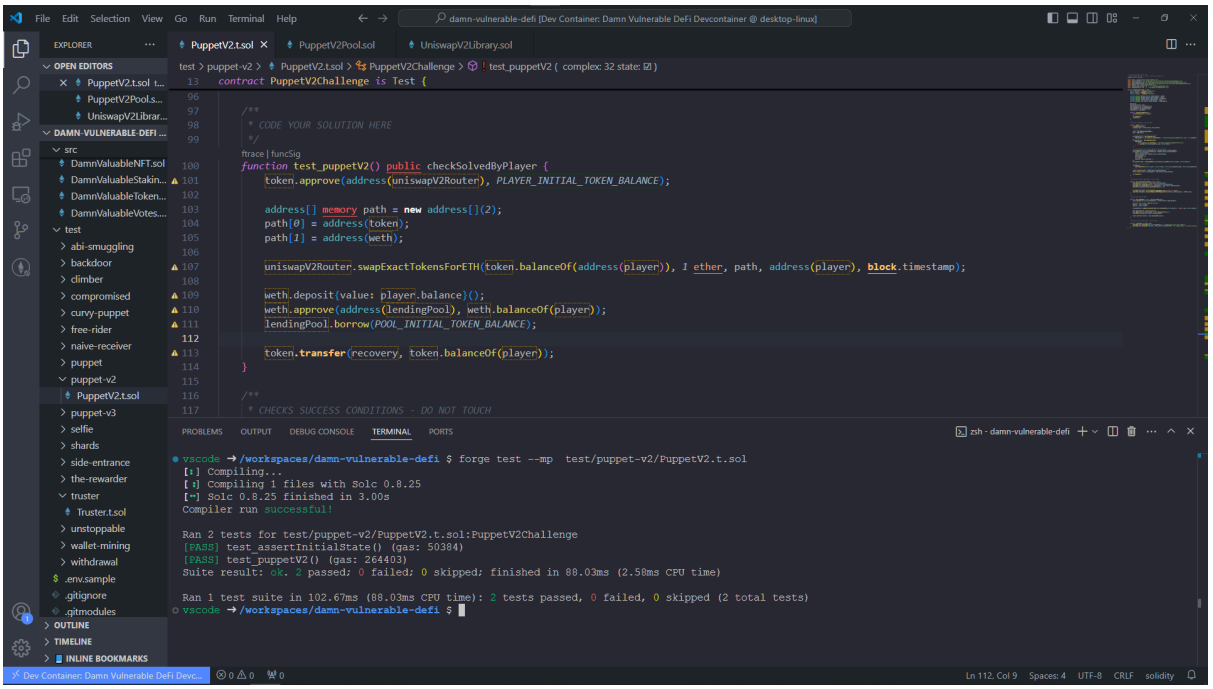
i. Mô tả lỗ hổng

- Challenge này gần giống như Challenge trước(Puppet). Tuy nhiên, họ bây giờ sử dụng Uniswap v2 và một vài thư viện được đề xuất
- Ngoài ra, lượng đặt cọc lần này là gấp 3
- Bạn có 20 ETH và 10000 DVT tokens. Pool có 1 triệu DVT tokens
- Giái cứu tất cả tiền và chuyển vào recovery account

ii. Khai thác lỗ hổng

- Chúng ta có thể thấy rằng chỉ cần tăng số DVT trong Uniswap V2 thì giá của nó sẽ giảm mạnh(hàm **quote)**
- Đầu tiên, ta đổi tất cả DVT của mình vào Uniswap V2 - làm tăng lượng DVT trong Uniswap V2. Điều này khiến cho giá của DVT giảm mạnh.

- Vì giá của DVT hiện tại rất thấp, ta có thể mượn tất cả DVT trong pool rồi chuyển cho recovery account



j. Câu 10: Free Rider

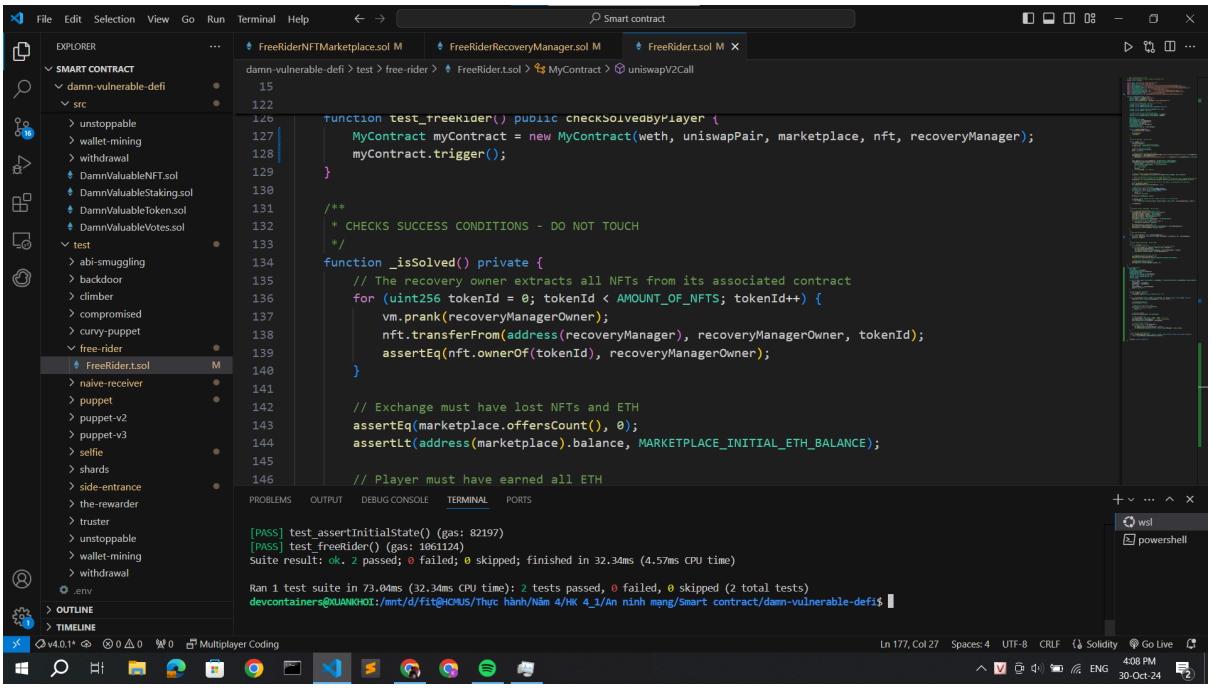
i. Mô tả lỗ hổng

- Một thị trường mới cho các NFT, hiện có 6 NFT đã được phát hành ban đầu, và mỗi NFT được bán với giá 15 ETH.
- Vấn đề với phương thức **buyMany**:
  + **buyMany** cho phép người dùng mua nhiều NFT trong một lần gọi, bằng cách gọi **_buyOne** lặp lại cho từng **tokenId** trong danh sách tokenIds.
  + Tuy nhiên, hợp đồng chỉ kiểm tra **msg.value** (số lượng ETH gửi vào) cho từng lần gọi **_buyOne**, mà không kiểm tra tổng giá trị cần trả cho toàn bộ các NFT.
  + Điều này có nghĩa là kẻ tấn công có thể mua nhiều NFT mà chỉ cần chi trả số ETH bằng giá của một NFT, vì mỗi lần gọi _buyOne chỉ yêu cầu đủ ETH cho một NFT mà không tính đến số lần gọi thêm.
- Cơ chế hoàn tiền không chính xác:
  + Trong **_buyOne**, sau khi NFT được chuyển cho người mua, **sendValue** sẽ hoàn lại ETH cho chủ sở hữu NFT.

+ Nhưng sau khi gọi **safeTransferFrom** để chuyển quyền sở hữu NFT cho người mua (trong trường hợp này là kẻ tấn công), người mua trở thành chủ sở hữu mới của NFT.
+ Do đó, khi **sendValue** được gọi, nó sẽ chuyển lại 15 ETH về cho chính kẻ tấn công thay vì trả cho người bán ban đầu.

ii. Khai thác lỗ hổng

- Đầu tiên, thực hiện lệnh **swap** của Uniswap để mượn tạm 15 ETH dưới dạng WETH (Wrapped ETH) từ Uniswap V2. Đây là số vốn cần thiết để mua tất cả 6 NFT, mỗi NFT có giá là 15 ETH.
- Thực Hiện Callback **uniswapV2Call**:
  + Tấn công với **buyMany** để mua tất cả 6 NFT, nhưng do lỗi trong hợp đồng, chỉ cần 15 ETH thay vì 90 ETH.
  + Sau mỗi lần mua, do hàm **sendValue** gửi lại tiền cho kẻ tấn công, kẻ tấn công nhận lại số tiền đã bỏ ra.
  + Sau khi nhận lại số tiền từ **buyMany**, kẻ tấn công chuyển đổi số tiền này thành WETH và hoàn trả khoản vay cho **Uniswap**, bao gồm cả phí 0.3%.
  + NFT được chuyển vào **recoveryManager** qua hàm **safeTransferFrom** để nhận phần thưởng
- Hoàn thành challenge:



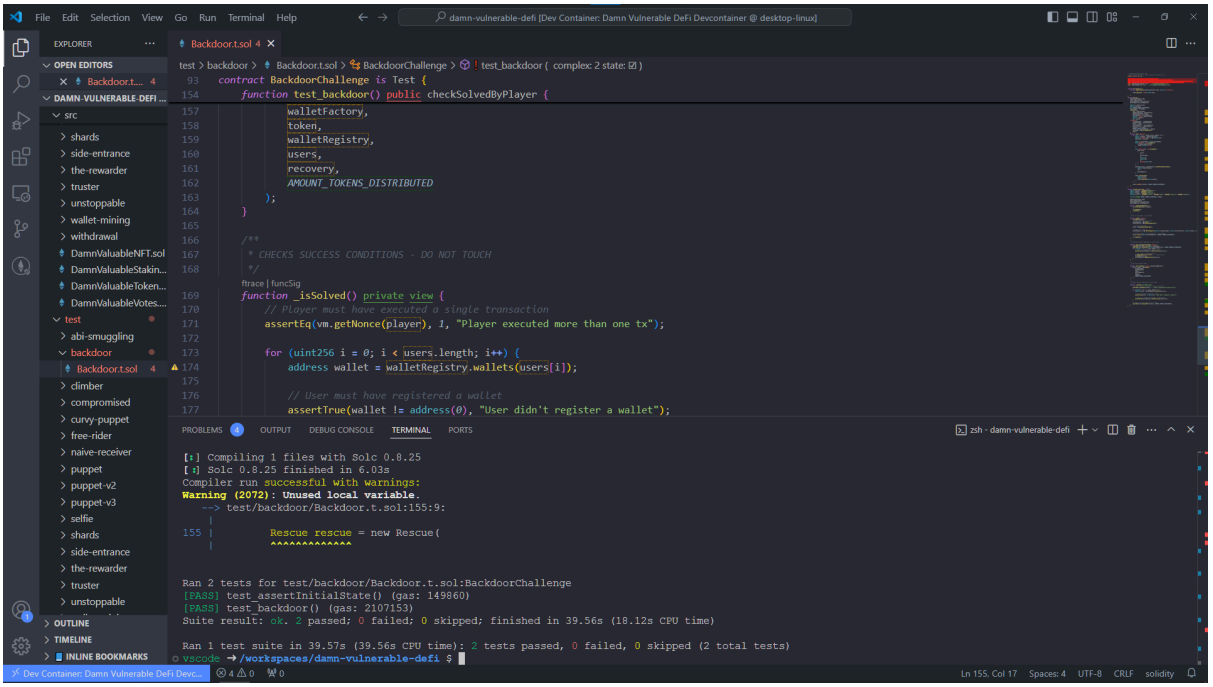
k. Câu 11: Backdoor

i. Mô tả lỗ hổng

- Để thúc đẩy việc tạo ra nhiều ví an toàn hơn. Có ai đó đã tạo ra một bản ghi các Safe wallets. Nếu có ai đó trong team tạo ra và đăng ký ví, người đó nhận được 10 DVT tokens.
- Bản ghi được tích hợp với Safe Proxy Factory - gồm nhiều phép kiểm tra an toàn
- Hiện tại có bốn người được gọi là những người được hưởng quyền lợi: Alice, Bob, Charlie, David. Bản ghi có 40 DVT tokens để được chia sẻ giữa họ.
- Giair cứu tất cả tiền vào recovery account trong một giao dịch

ii. Khai thác lỗ hổng

- Ta thấy vấn đề nằm ở hàm **setup** của Safe contract. Nó cho phép delegateCall đến bất cứ contract nào.
- Đầu tiên, ta tạo một **contract** để approve tất cả DVT cho người chơi
- Khi thiết lập ví thì chúng ta sẽ truyền tham số vào hàm setup để gọi **contract** trên.
- Khi ta nhận được 10 DVT thì ta gửi đến **contract** đó
- Thực hiện những bước trên cho tất cả những người được hưởng quyền lợi. Sau đó gửi DVT từ **contract** vào recovery account
- Tất cả các thao tác trên đều được thực hiện khi constructor của **contract** đó được gọi. Làm vậy để thực hiện tất cả trong 1 giao dịch.



I. Câu 12: Climber

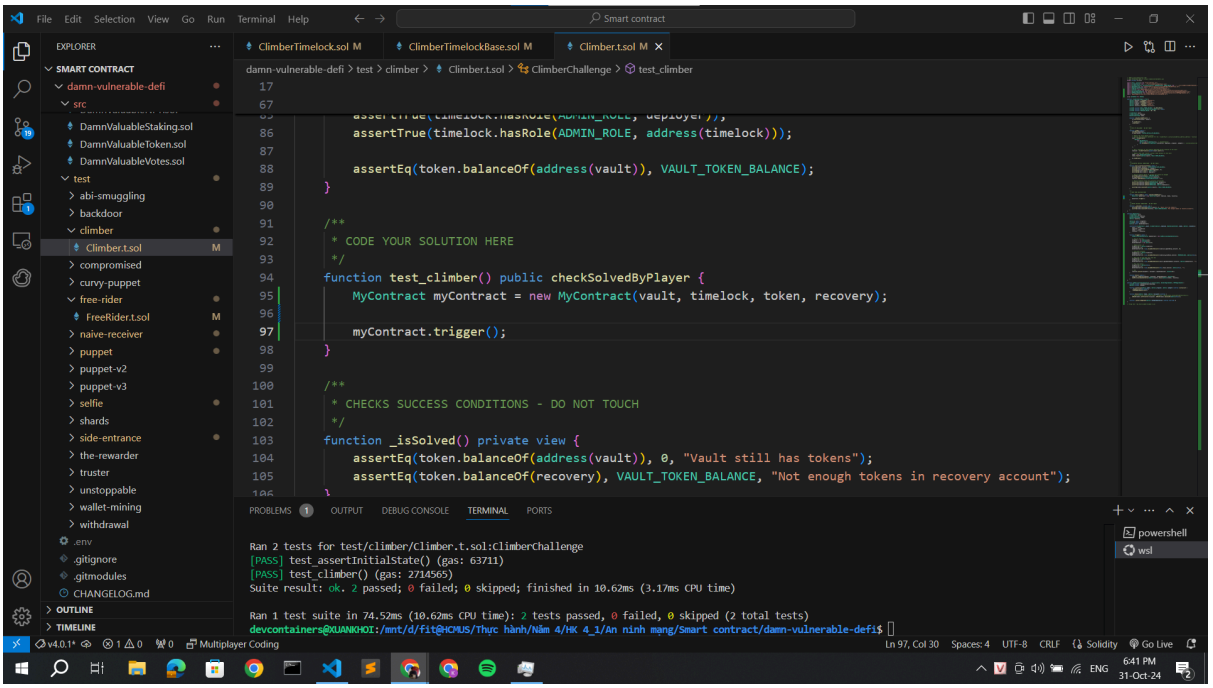
i. Mô tả lỗ hổng

- Ta có 1 vault chứa 10 triệu DVT token và 1 contract **timelock** chỉ cho phép rút 1 lượng token trong 15 ngày
- Trên **timelock**, chỉ có một tài khoản với vai trò Proposer mới có thể lập lịch cho các hành động, và các hành động này có thể được thực hiện sau 1 giờ.
- Hàm **Execute**
    + Kiểm tra trạng thái thời gian được thực hiện sau khi thực thi các hành động.
    + Điều này có nghĩa là chúng ta có thể thực hiện hành động trước và sau đó kiểm tra thời gian.
    + Nếu hành động đó không bị cản trở bởi trạng thái thời gian, thì các hành động sẽ vẫn được thực hiện.
- Lợi dụng **updateDelay**
    + Chúng ta có thể thay đổi độ trễ thành 0

ii. Khai thác lỗ hổng

- Do không thể trở thành sweeper để lấy tiền, chúng ta sẽ cấp quyền cho chính mình với vai trò **PROPOSER_ROLE** và tạo một purposal mới.

- Contract vault sử dụng mẫu nâng cấp UUPS, do đó chúng ta có thể nâng cấp contract này đến một địa chỉ mới mà chúng ta chỉ định (a new contract).
- Khi nâng cấp, contract mới sẽ cần có hàm khởi tạo và hàm **_authorizeUpgrade** theo mẫu UUPS.
- Trong contarct mới, chúng ta sẽ cung cấp một hàm **drain** để chuyển toàn bộ tiền đến recovery
- Hoàn thành challenge:



m. Câu 13: Wallet Mining

i. Mô tả lỗ hổng

- Một contract khuyến khích người dùng tạo ví **Safe**, người tạo sẽ được tặng 1 DVT. Nó được tích hợp cơ chế bảo mật(nâng cấp được), chỉ cho phép một số deployers(wards) được nhận tiền trong một số trường hợp nhất định
- Deployer Contract chỉ hoạt động với **Safe factory** và copy set khi đang được triển khai. Có vẻ như là **Safe singleton factory** đã được triển khai
- Họ chuyển 20 triệu DVT tokens đến user với địa chỉ là 0x8be6a88D3871f793aD5D5e24eF39e1bf5be31d2b - chỗ mà

1-of-1 **Safe** được cho rằng sẽ ở đó. Nhưng họ đã mất nonce mà họ nên dùng cho việc triển khai

- Còn có một tin đồn rằng đang có lỗ hổng trong hệ thống. Bạn được quyền truy cập đến private key của cô ấy
- Giải cứu tất cả tiền
- Phục hồi tất cả tokens ở wallet deployer contract và gửi đến wards tương ứng. Trả tất cả tiền về cho user.
- Làm tât cả trong một giao dịch

ii. Khai thác lỗ hổng

- Tìm ra Nonce bằng cách Loop cho đến khi nó trùng với **USER_DEPOSIT_ADDRESS**
- Gọi hàm **walletDeployer.drop()**, dùng **createProxyWithNonce** để tạo ra ví của người dùng
- Contract **AuthorizerUpgradeable** chiếm slot0 với biến needInit. Điều này sẽ dẫn đến việc trùng địa chỉ khi tạo ví. Từ đó ta có thể biến chúng ta thành wards.
- Chuyển tiền token từ contract vào wards

n. Câu 14: Puppet V3

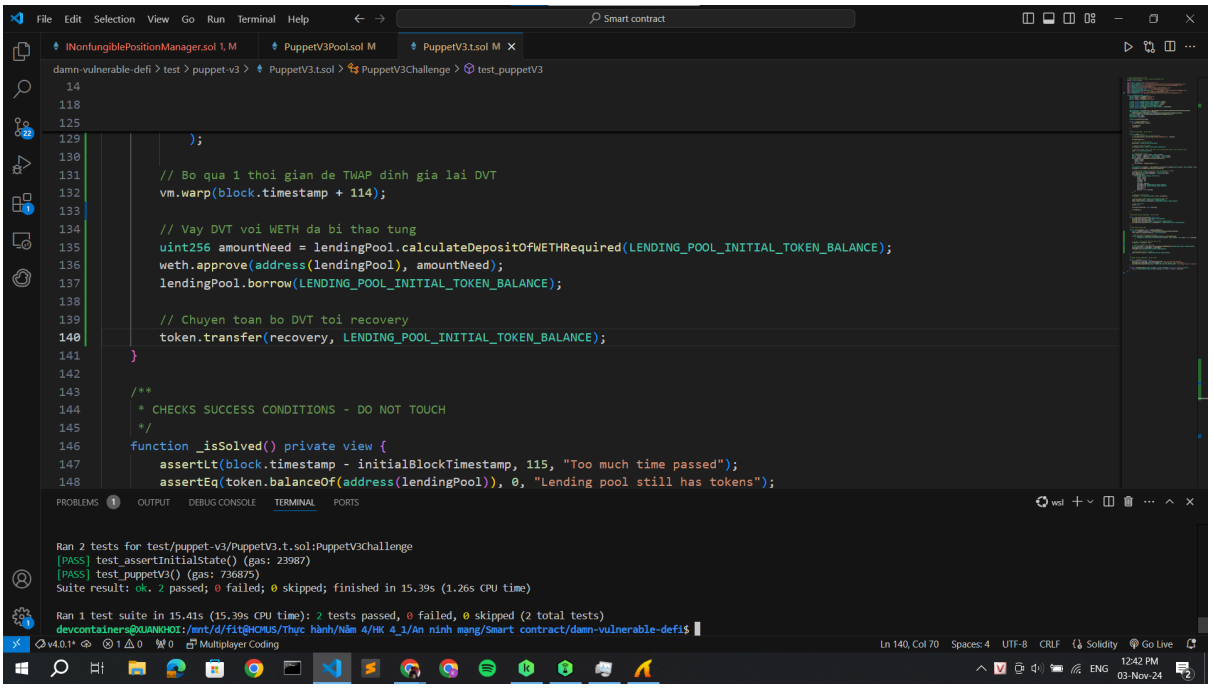
i. Mô tả lỗ hổng

- Khác với Puppet v1, phiên bản V3 này sử dụng *time-weighted average price (TWAP)* thay vì giá spot
- Pool Uniswap V3 có 100 WETH và 100 DVT, tạo điều kiện thuận lợi cho việc thao túng giá.
- Thực hiện giao dịch lượng lớn DVT để đổi lấy WETH, điều này sẽ khiến giá DVT giảm mạnh. Bởi TWAP lấy giá trung bình từ khoảng thời gian 10 phút, giao dịch lớn sẽ làm ảnh hưởng nặng nề đến giá.
- Khai Thác Độ Trễ của TWAP:
  + PuppetV3Pool có một khoảng thời gian 110 giây, trong đó TWAP chưa cập nhật giá hoàn toàn.
  + Trong khoảng thời gian này, ta có thể thực hiện các thao tác vay DVT dựa trên giá đã bị thao túng trước khi TWAP điều chỉnh lại giá về trạng thái bình thường.

ii. Khai thác lỗ hổng

- Đầu tiên, ta sẽ thao túng giá: Swap DVT cho WETH trên Uniswap qua **ISwapRouter**, làm cho giá DVT bị giảm so với WETH.

- Tận dụng TWAP delay: Sau khi swap, tiến hành **warp** thêm 114 giây để chờ TWAP cập nhật, nhưng vẫn trong khoảng thời gian giới hạn (đề giới hạn dưới 115 giây)
- Dùng WETH làm tài sản thế chấp để vay toàn bộ DVT với số lượng tài sản thế chấp ít hơn do giá đã bị thao túng.
- Chuyển DVT về tài khoản **recovery**
- Hoàn thành challenge:



o. Câu 15: ABI Smuggling

i. Mô tả lỗ hổng

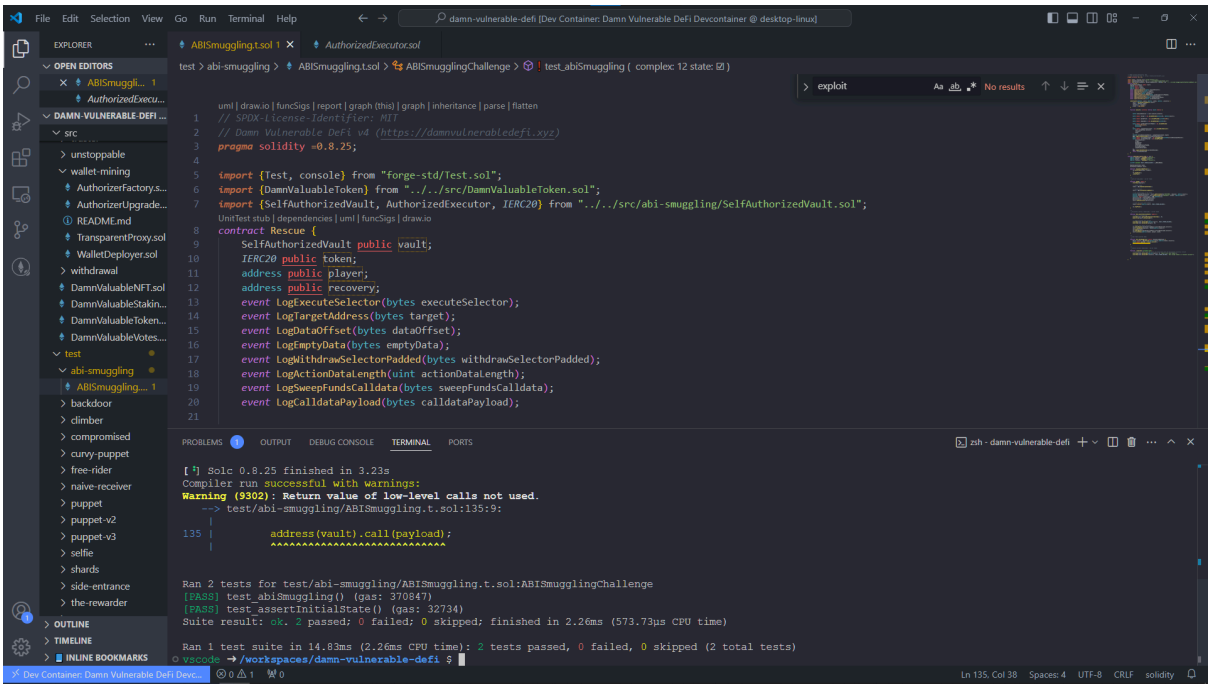
- Có một cái kho không cần cấp quyền với 1 triệu DVT token. Kho cho phép rút tiền theo giai đoạn, và cho phép lấy tất cả tiền ra trong tường hợp khẩn cấp.
- Contract có hệ thống kiểm tra quyền, chỉ cho phép những tài khoản quen thuộc thực hiện một số thao tác nhất định
- Dev team được tiết lộ rằng tất cả tiền có thể bị trộm
- Hãy giải cứu tất cả tiền ra khỏi đó và gửi vào recovery account

ii. Khai thác lỗ hổng

- Trong hàm AuthorizedExecutor.execute(), lệnh calldataload được sử dụng để trích xuất 4 byte của function selector từ actionData

bắt đầu từ calldataOffset (100 byte), sau đó kiểm tra xem ID này có được cấp phép thông qua getActionId hay không.

- Người triển khai hợp đồng có thể thực thi sweepFunds với selector 0x85fb709d, và người chơi có thể thực thi withdraw với selector 0xd9caed12.
- Mấu chốt là vượt qua kiểm tra getActionId, điều này cho phép thực thi functionCall một cách tùy ý.



p. Câu 16: Shards

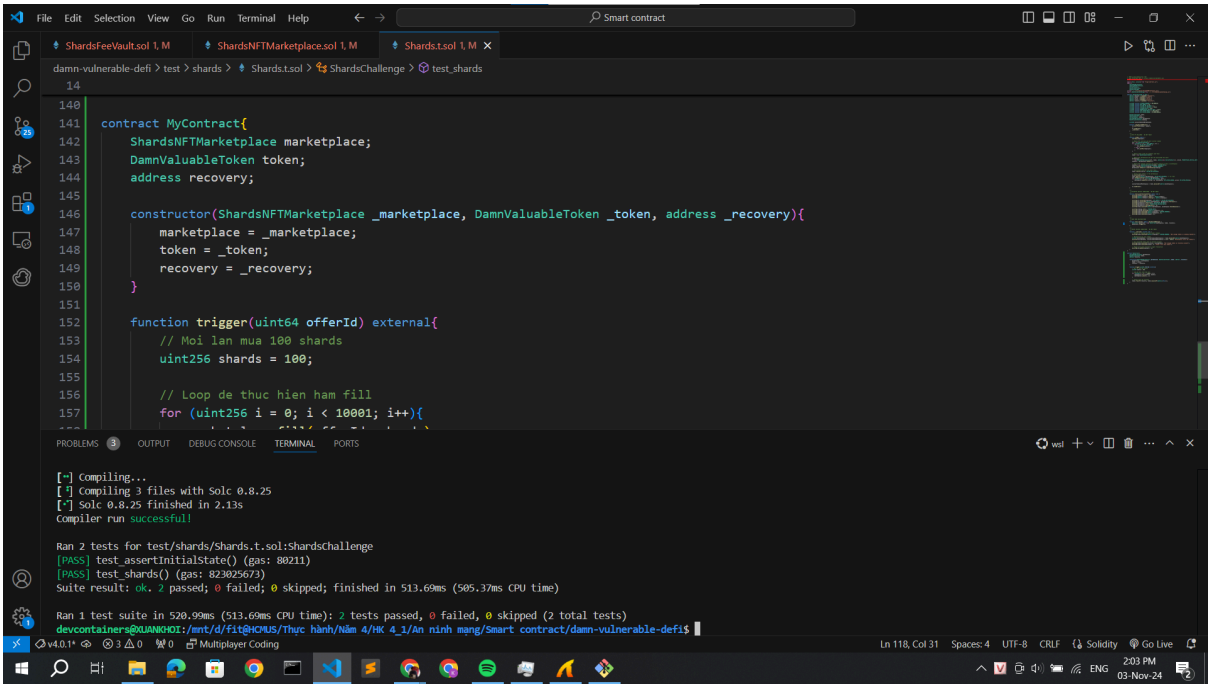
i. Mô tả lỗ hổng

- Một thị trường NFT có tên là Shards, cho phép người sở hữu NFT bán chúng dưới dạng từng phần nhỏ (shards) và nhận thanh toán bằng USDC.
- Thị trường này không yêu cầu quyền truy cập, cho phép những người nắm giữ NFT Damn Valuable bán chúng với bất kỳ mức giá nào (được biểu thị bằng USDC).
- Các NFT này có thể có giá trị cao đến mức người bán có thể cung cấp chúng dưới dạng những phần nhỏ hơn (“shards”).
- Lỗ hổng:
    + Khi kiểm tra hàm **fill()**, phát hiện rằng **want.mulDivDown(_toDVT(offer.price, _currentRate), offer.totalShards)** tính toán số lượng shards mà người mua có thể mua dựa trên giá trị want.

+ Tuy nhiên, phép toán trong hàm này có thể gặp phải lỗi tràn hoặc lỗi tính toán, đặc biệt với sự kết hợp của **mulDivDown** và **_toDVT**.
+ Thuật toán này khiến cho kết quả cuối cùng trở thành 0 khi giá trị want là một giá trị nhỏ.
+ Giá trị tối đa của **want** mà có thể dẫn đến việc mua với giá 0 là 133.
+ Sử dụng các shards NFT không mất phí này, ta có thể gọi hàm **cancel()** để trả lại shards cho thị trường, và vào lúc này, bạn sẽ nhận được token DVT.

ii. Khai thác lỗ hổng

- Ta sẽ thực hiện một cuộc tấn công vào marketplace bằng cách lợi dụng việc cung cấp và hủy bỏ các giao dịch mua shards NFT
- Ta sẽ thực hiện 1 vòng lặp:
  + Gọi hàm **fill**: với **offerId** và **wantShards**, có nghĩa là người tấn công đang cố gắng mua 100 shards.
  + Gọi hàm **cancel**: Sau khi thực hiện fill, hàm cancel được gọi với hai tham số: ID và biến i trong vòng lặp. Điều này có thể được sử dụng để hủy bỏ giao dịch hoặc hoàn lại một phần hoặc toàn bộ các shards mà người tấn công đã mua, giúp người tấn công lấy lại số lượng token mà họ đã sử dụng để thực hiện giao dịch mà không mất phí.



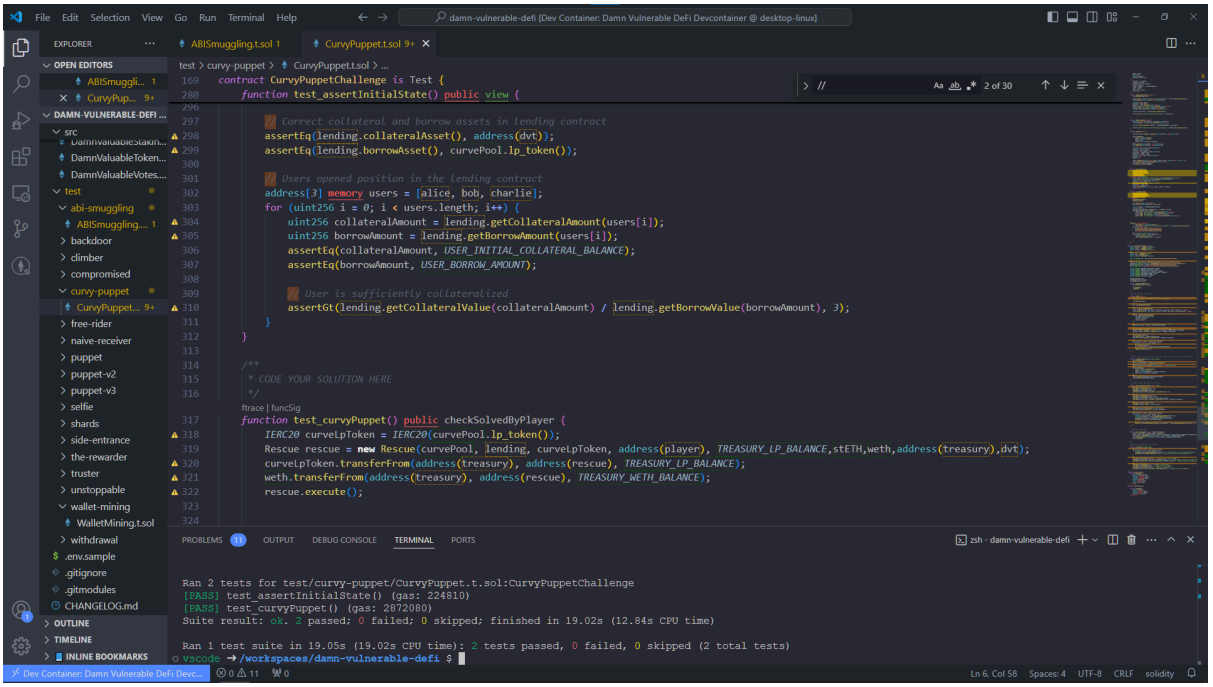
q. Câu 17: Curvy Puppet

i. Mô tả lỗ hổng

- Có một hợp đồng cho vay cho phép bất kỳ ai có thể vay các token LP từ pool stETH/ETH của Curve. Để làm như vậy, người vay phải nạp một lượng token Damn Valuable Token (DVT) làm tài sản thế chấp.
- Hợp đồng này sử dụng Permit2 để quản lý quyền phê duyệt token một cách an toàn và một oracle để lấy giá của ETH và DVT.
- Oracle này có thể là điểm yếu nếu giá được thao túng để lợi dụng cơ chế thanh lý.
- Mục tiêu là tìm cách đóng các vị thế của Alice, Bob, và Charlie để cứu lấy tài sản thế chấp của họ trước khi khoản tiền của người dùng bị mất. Điều này thường yêu cầu đẩy giá trị tài sản thế chấp xuống so với khoản vay của họ để có thể kích hoạt việc thanh lý.

ii. Khai thác lỗ hổng

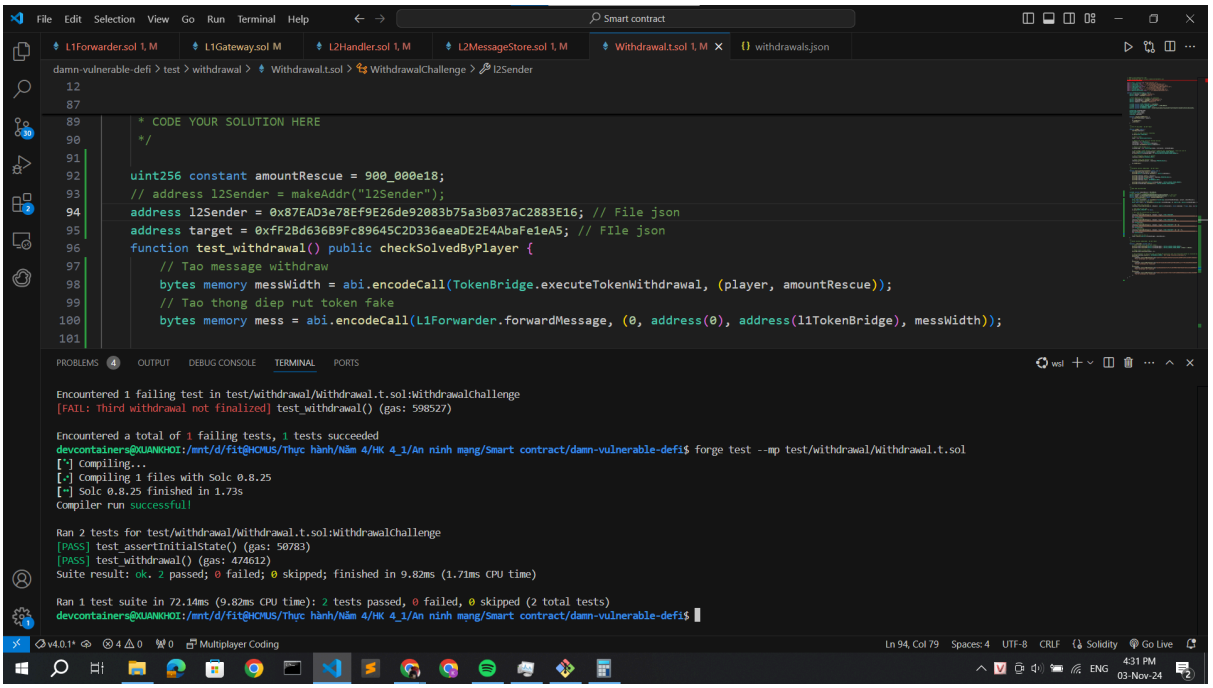
- Để thanh lý, điều kiện if (collateralValue >= borrowValue) phải revert HealthyPosition(borrowValue, collateralValue).
- Cuối cùng, tôi đã thành công bằng cách sử dụng hai flashloan để hoàn thành thử thách này.
- Điều quan trọng là Balancer cho phép vay WETH mà không có phí. Điều này giúp tôi tính toán được một khoản đủ để thanh lý, đồng thời vẫn có đủ quỹ để hoàn trả flashloan.



r. Câu 18: Withdrawal

i. Mô tả lỗ hổng

- Ta có 1 cầu nối (bridge) từ L2 sang L1 để rút DVT.
- Phía L1 cho phép bất kỳ ai cũng có thể hoàn tất việc rút tiền, miễn là thời gian trì hoãn đã trôi qua và họ cung cấp một bằng chứng Merkle hợp lệ.
- File JSON cung cấp nhật ký sự kiện của 4 giao dịch rút tiền được khởi tạo trên L2. Chúng có thể được thực hiện trên L1 sau thời gian trì hoãn 7 ngày.
- Điều kiện chiến thắng:
    + Cầu nối Token L1 phải giữ ít nhất 99% số token.
    + Số dư token của người chơi phải là 0.
    + Giá trị của **counter()** trong L1 Gateway phải lớn hơn hoặc bằng **WITHDRAWALS_AMOUNT**, điều này cho thấy rằng đã hoàn thành đủ số lượng giao dịch rút tiền.
    + Quy trình rút tiền: Để xử lý các yêu cầu rút tiền, sử dụng hàm **L1Gateway.finalizeWithdrawal**. Khi người gọi là một Operator, hợp đồng sẽ không kiểm tra MerkleProof, cho phép ta giả mạo yêu cầu và rút token từ cầu nối.
    + Lợi dụng quyền **Operator**, có thể thực hiện yêu cầu rút 900,000 token (không được rút hết) từ L1.

ii. Khai thác lỗ hổng

- Đầu tiên, ta sẽ thực hiện giao dịch “giả” với quyền **Operator** để rút 900,000 token với mục tiêu là L1 và trong vòng 7 ngày
- Sau đó, ta phải thực hiện bốn giao dịch theo yêu cầu của bài toán kèm với **Merkle proof**
- Cuối cùng, ta chuyển tiền trở lại L1
- Hoàn thành challenge: